

PyTorch & CNNs Crash Course

Tensors, autograd, nn.Module, and convolutional networks

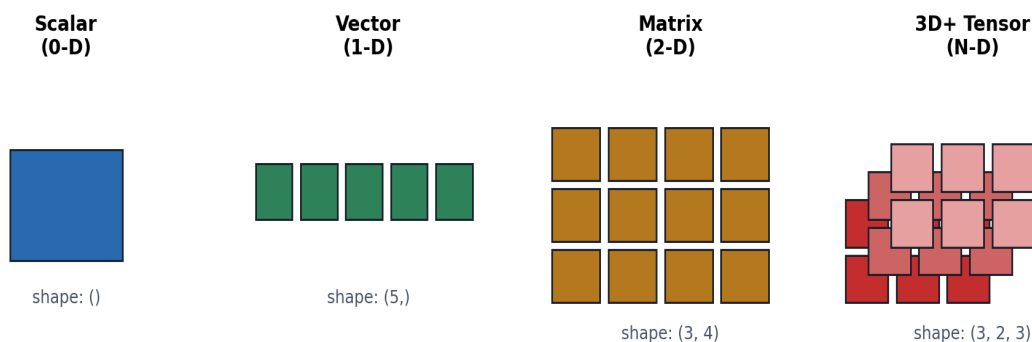
1. What Is PyTorch?

PyTorch is a deep learning framework that automates the two most tedious and error-prone parts of building a neural network by hand: computing gradients (backpropagation) and applying parameter updates. Everything you implemented manually in NumPy — forward propagation, the chain rule, gradient descent — still happens exactly the same way underneath; PyTorch just tracks and executes it for you.

Three pieces do almost all of the work: tensors (the data), `nn.Module` (the model architecture), and `autograd` (automatic gradient computation).

2. Tensors

A tensor is PyTorch's core data structure — conceptually the same as a NumPy array, generalized to any number of dimensions, with one crucial addition: a tensor can track every operation performed on it, which is what makes automatic differentiation possible.



A single number is a 0-D tensor (scalar); a list of numbers is 1-D (vector); a table is 2-D (matrix); anything with more dimensions — like a batch of images — is an N-D tensor.

- **Example** — A single MNIST image (28x28 pixels, 1 color channel): shape `(1, 28, 28)`
- **Example** — A batch of 64 MNIST images: shape `(64, 1, 28, 28)`
- **Example** — A weight matrix connecting a 784-input layer to a 64-neuron hidden layer: shape `(784, 64)` — exactly your `W1` from the from-scratch notebook

3. `nn.Module` — Defining a Model

Every PyTorch model is a Python class inheriting from `nn.Module`. Two parts matter:

- **`__init__`** — Define your layers as objects (e.g. `nn.Linear(784, 64)`) — each one owns and manages its own weights and biases automatically, so you never manually create or track a `W/b` matrix again.
- **`forward()`** — Define how data actually flows through those layers, in order. This is your `forward_propagation` function, but you only write the layer calls — no manual matrix multiplication.

```
class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        return self.fc2(x)
```

4. Autograd — Automatic Backpropagation

Autograd is the mechanism that makes `.backward()` work. As data flows through your model during the forward pass, PyTorch silently builds a computation graph — a record of every operation performed. Calling `loss.backward()` walks that graph in reverse, applying the chain rule at every step, and computes the gradient of the loss with respect to every parameter in the model — automatically.

This is exactly the process you implemented by hand: propagating error backward through each layer, using the chain rule. Autograd does not use a different method — it performs the same computation, just without you writing out each derivative yourself.

```
loss.backward() # computes every gradient in the model, via the chain rule
optimizer.step() # applies those gradients to update every parameter
optimizer.zero_grad() # clears gradients before the next batch (PyTorch accumulates by default)
```

5. Training Loop Structure

The overall shape hasn't changed since your very first linear regression notebook: initialize, loop, forward pass, compute loss, backward pass, update parameters. PyTorch just automates the middle two steps.

```
for epoch in range(n_epochs):
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

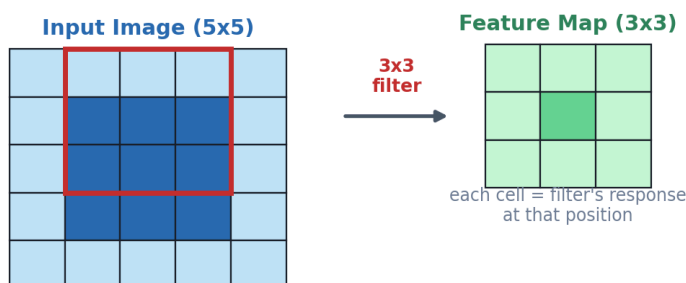
One genuinely new idea here: mini-batch training. Your from-scratch notebook used full-batch gradient descent — the entire training set, every iteration. A `DataLoader` instead splits data into small batches (e.g. 64 images at a time), and one full pass through all batches is called an epoch. Mini-batches are standard practice: faster per step, and the added randomness often improves training stability.

6. Convolutional Neural Networks (CNNs)

A plain fully-connected network treats every pixel as an independent feature, ignoring that nearby pixels are spatially related — two adjacent pixels being part of the same edge is meaningful information a flat vector throws away. CNNs are built specifically to exploit that spatial structure.

Convolution

A convolutional layer slides a small filter (e.g. 3×3) across the image. At every position, the filter computes a weighted sum of the pixels underneath it, producing one value in the output feature map. The SAME filter weights are reused at every position — this is what makes CNNs dramatically more parameter-efficient than a fully-connected layer of comparable size.



A 3×3 filter slides across the input image. Each position produces one value in the output feature map, representing how strongly that filter's pattern was detected there.

Pooling

A pooling layer downsamples a feature map — typically by taking the maximum value in each small region (max pooling). This shrinks the spatial size while keeping the strongest activations, reducing computation and making the network somewhat robust to small shifts in where a pattern appears in the image.

A typical CNN architecture



Convolution and pooling layers alternate, progressively extracting more abstract features while shrinking spatial size, before flattening into fully-connected layers for the final classification.

Why fewer parameters than a fully-connected layer: a 3x3 filter with 16 output channels on a single-channel image has only $3 \times 3 \times 1 \times 16 = 144$ weights, reused across the ENTIRE image. A fully-connected layer mapping the same 784 input pixels to just 64 outputs needs $784 \times 64 = 50,176$ weights — with a separate, unshared weight for every single pixel position.

7. Quick Glossary

Term	Plain-English Meaning
Tensor	PyTorch's core data structure — a NumPy-array-like container that can hold data of any dimensionality, and additionally tracks gradients for autograd.
nn.Module	The base class every PyTorch model inherits from. You define layers in <code>__init__</code> and how data flows through them in <code>forward()</code> .
nn.Linear	A fully-connected layer — owns its own weight matrix and bias, equivalent to one W/b pair from a from-scratch implementation.
Autograd	PyTorch's automatic differentiation engine. It tracks every operation performed on tensors so it can compute gradients automatically via <code>.backward()</code> .
Logits	A model's raw, un-normalized output scores before an activation like softmax is applied.
nn.CrossEntropyLoss	The standard multi-class loss function — applies softmax internally and computes categorical cross-entropy against the true labels.
Optimizer	An object (e.g. SGD, Adam) that holds the update rule and applies it to every model parameter when <code>.step()</code> is called.
optimizer.zero_grad()	Clears old gradients before a new backward pass — PyTorch accumulates gradients by default, so skipping this causes stale gradients to pile up.
DataLoader / batch	Splits a dataset into small chunks (batches) fed to the model one at a time per training step, rather than the whole dataset at once.
Epoch	One full pass through the entire training dataset (potentially many batches/iterations).
Convolution / filter	A small learned grid of weights slid across an image, detecting a specific local pattern (edge, curve, texture) wherever it appears.
Feature map	The output produced by sliding one filter across an entire image — shows where that filter's pattern was detected.
Pooling	Downsamples a feature map (e.g. taking the max value in each small region), shrinking spatial size while keeping the strongest signals.

8. Where This Connects to Your Roadmap

Your Part 2 notebook builds a feedforward network deliberately mirroring your from-scratch architecture, so you can map `optimizer.zero_grad() / loss.backward() / optimizer.step()` directly onto the manual code you already wrote and debugged in Part 1 — including the same instability lessons (learning rate, initialization) that autograd doesn't exempt you from, it just handles the arithmetic. The CNN section then extends into genuinely new territory: exploiting the 2D spatial structure of images, which a flat-vector fully-connected network cannot do.